```c
/*
 * @file    Ejercicio_8_2.c
 * @brief   Realiza la generación de una forma de onda desde un buffer específico
 *          en la AHB SRAM a través del módulo DAC, de manera indefinida mediante
 *          una LLI circular. Monitoreando el estado de la tranferencia mediante
 *          interrupción del GPDMA.
 * @details Configuración del Periférico DAC para Transferencia M2P con LLI
 *          Simple e ISR vía GPDMA
 * @author  Ing. E. Migliore
 * @date    30/04/2026
 * @version 2.0
 */

/* Directivas de Inclusión */
#include "LPC17xx.h"
#include "lpc17xx_gpdma.h"

/* Definición de Constantes y Macros */
#define DAC_BUFFER_START 0x20080000     //WaveForm
#define NUM_WAVE_SAMPLES 382

/* Definición de Variables Globales */
volatile uint32_t *dac_samples = (volatile uint32_t *)DAC_BUFFER_START;
volatile uint32_t DMA_CH0_TC = 0;   //Flag de Cuenta Terminal para Canal 0 de DMA
volatile uint32_t DMA_CH0_Err = 0;  //Flag de Error Terminal para Canal 0 de DMA
volatile bool DAC_FLAG = true;

/* Prototipos de Funciones */
void cfgDMA(void);
void loadDACwave(void);

/* Función Principal  */
int main(void)
{
    cfgDAC();
    cfgDMA();

    //Trigger por software
    while(1) if(DAC_FLAG) { loadDACwave(); DAC_FLAG = false; }

    return 0;
}

/* Definiciones de Funciones */
void loadDACwave(void)
{
    //Inicialización de Envió de Datos por Canal 0 de DMA
    GPDMA_ChannelCmd(LPC_GPDMACH0, ENABLE);
    //Habilitación de Interrupción por DMA
    NVIC_EnableIRQ(DMA_IRQn);
    return;
}

void cfgDAC(void){
    //Número de Cuentas para el Timer Dedicado al DAC
    uint32_t countDAC = 65535;
    DAC_CONVERTER_CFG_Type cfgDAC;

    //Habilitación de TIMER Dedicado al DAC
    cfgDAC.CNT_ENA = SET;
    //Habilitación de DAC por DMA
    cfgDAC.DMA_ENA = SET;

    //Inicialización del DAC
    DAC_Init(LPC_DAC);
    //BIAS=1: Tiempo de Establecimiento de 2.5us (max)
    DAC_SetBias(LPC_DAC,DAC_MAX_CURRENT_350uA);

    //Configuración del DAC
```

```
70      DAC_ConfigDAConverterControl(LPC_DAC, &cfgDAC);
71      //*** CNT_ENA=1: Habilitación de TIMER Dedicado al DAC ***
72      //LPC_DAC->DACCTRL |= (1<<2);
73      //*** DMA_ENA=1: Habilitación de DAC por DMA ***
74      //LPC_DAC->DACCTRL |= (1<<3);
75
76      //Establecimiento del Valor de Recarga para el Timer Dedicado
77      DAC_SetDMATimeOut(LPC_DAC,countDAC);
78      //*** DACCNTVAL=65535[cuentas] ***
79      //LPC_DAC->DACCNTVAL = (countDAC&0xFFFF);
80      return;
81  }
82
83  void cfgDMA(void)
84  {
85      //Estructura para Lista Enlazada para canal 0 del DMA
86      static GPDMA_LLI_Type cfgDAC_LLI0_CH0;
87      //Estructura de Configuración para canal 0 del DMA
88      GPDMA_Channel_CFG_Type cfgDAC_DMA_CH0;
89
90      //Deshabilitación de Interrupción por DMA
91      NVIC_DisableIRQ(DMA_IRQn);
92      //Inicialización del Controlador GPDMA
93      GPDMA_Init();
94
95      //Dirección de Fuente de Datos (Memoria)
96      cfgDAC_LLI0_CH0.SrcAddr = (uint32_t)dac_samples;
97      //Dirección de Destino de Datos (Periferico)
98      cfgDAC_LLI0_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR);
99      //Próxima Dirección de Destino de Datos: LLI Repetitiva
100     cfgDAC_LLI0_CH0.NextLLI = (uint32_t)&cfgDAC_LLI0_CH0;
101     //Tamaño de la Transferencia de la LLI
102     cfgDAC_LLI0_CH0.Control = (NUM_WAVE_SAMPLES<<0)
103     //Ancho de la Transferencia de Fuente (32-bit)
104                             |(2<<18)
105     //Ancho de la Transferencia de Destino (32-bit)
106                             |(2<<21)
107     //SI=1: Con Incremento de la Dirección de Fuente
108                             |(1<<26)
109     //DI=0: Sin Incremento de la Dirección de Destino
110                             & ~(1 << 27);
111
112     //Selección del Canal de Transferencia
113     cfgDAC_DMA_CH0.ChannelNum = 0;
114     //Periférico de Fuente de Datos (Sin Uso)
115     cfgDAC_DMA_CH0.SrcConn = 0;
116     //Periférico de Destino de Datos (DAC)
117     cfgDAC_DMA_CH0.DstConn = GPDMA_CONN_DAC;
118     //Dirección de Fuente de Datos en Memoria
119     cfgDAC_DMA_CH0.SrcMemAddr = (uint32_t)dac_samples;
120     //Dirección de Destino de Datos en Memoria (Sin Uso)
121     cfgDAC_DMA_CH0.DstMemAddr = 0;
122     //Tipo de Transferencia (Memoria a Periférico)
123     cfgDAC_DMA_CH0.TransferType = GPDMA_TRANSFERTYPE_M2P;
124     //Tamaño de Transferencia de Datos
125     cfgDAC_DMA_CH0.TransferSize = NUM_WAVE_SAMPLES;
126     //Ancho de Transferencia de Datos (Sin Uso)
127     cfgDAC_DMA_CH0.TransferWidth = 0;
128     //Lista Enlazada Inicial
129     cfgDAC_DMA_CH0.DMALLI = (uint32_t)&cfgDAC_LLI0_CH0;
130     //Configuración del Canal 0 de DMA
131     GPDMA_Setup(&cfgDAC_DMA_CH0);
132
133     return;
134 }
135
136 void DMA_IRQHandler (void){
137     //Verifica Interrupción por Canal 0 de DMA
138     if(GPDMA_IntGetStatus(GPDMA_STAT_INT, 0)){
```

```
139       //if(LPC_GPDMA->DMACIntStat & ((1<<0)&0xFF)){
140           //Verifica Interrupción por Cuenta Terminal (1 List)
141           if(GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 0)){
142           //if(LPC_GPDMA->DMACIntTCStat & ((1<<0)&0xFF)){
143
144               //Ejecuta ISR: DMA_CH0_TC++;
145
146               //Limpia Bandera de Interrupción por Cuenta Terminal
147               GPDMA_ClearIntPending (GPDMA_STATCLR_INTTC, 0);
148               //LPC_GPDMA->DMACIntTCClear |= ((1<<0)&0xFF);
149           //}
150           }
151           //Verifica Interrupción por Error Terminal
152           if(GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 0)){
153           //if(LPC_GPDMA->DMACIntErrStat & ((1<<0)&0xFF)){
154
155               //Ejecuta ISR: DMA_CH0_Err++;
156
157               //Limpia Bandera de Interrupción por Error Terminal
158               GPDMA_ClearIntPending (GPDMA_STATCLR_INTERR, 0);
159               //LPC_GPDMA->DMACIntErrClear |= ((1<<0)&0xFF);
160           //}
161           }
162        //}
163        }
164        return;
165   }
166
167
```